AYDT — Security & Vulnerability Audit

> **Purpose of this document:** This is the **security audit and production readiness assessment**. It catalogs every known vulnerability, explains WHY each is a vulnerability, tracks resolution status, and provides a prioritized remediation plan. For the system reference (what exists and how it works), see ARCHITECTURE_FULL.md (./ARCHITECTURE_FULL.md).

Audit Date: March 23, 2026

Previous Audit: February 24, 2026 (original), March 16, 2026 (update), March 22, 2026 (V-1–V-5 identified)

Last Remediation: March 23, 2026 (V-1 through V-5 resolved)

Executive Summary

AYDT has matured significantly since the February audit. The two original CRITICAL risks (payment stub, discount engine missing) are fully resolved. The EPG payment integration demonstrates strong security practices: timing-safe webhook auth, idempotency guards, server-only credentials, and a "never trust the webhook payload" verification pattern.

The March 22 audit uncovered critical vulnerabilities: **unauthenticated API endpoints** and **missing Row-Level Security on 9 database tables**. As of March 23, all five CRITICAL/HIGH pre-audit items (V-1 through V-5) have been remediated:

- **V-1 RESOLVED**: All 6 media/upload API routes now require admin authentication via requireAdmin()
- **V-2 RESOLVED**: RLS enabled on all 9 tables with role-based policies (admin, parent, instructor). instructor role added to the system with instructor_id FK on class_sessions
- **V-3 RESOLVED**: Server-side admin route protection added in middleware (previously client-side only)
- **V-4 RESOLVED**: Webhook signature verification is now mandatory for both Resend and Twilio (no silent fallback)
- **V-5 RESOLVED**: Payment amount is now re-validated from the server-stored registration_batches.amount_due_now — client input is never trusted

Bottom line for client audit readiness (~April 2, 2026):

- All CRITICAL pre-audit items are resolved.
- The HIGH items (V-6 through V-12) should be fixed before production launch.
- MEDIUM and LOW items are hardening work that can happen post-launch.

Part 1 — Resolution Status of February 2026

Findings

#	Original Finding	Severity	Status	Notes
R1	Payment system is a complete stub	CRITICAL	**RESOLVED**	Full EPG integration (hosted checkout, stored cards/ACH, recurring)
R2	Discount computation not implemented	CRITICAL	**RESOLVED**	Tuition engine + coupon system implemented
R3	Email status marked "sent" before dispatch	CRITICAL	**RESOLVED**	Now uses "sending" intermediate status; marks "sent" only after broadcast confirms
R4	Broadcast emails send empty student/semester/session names	HIGH	**OPEN**	send-email-broadcast/index.ts:118-120 still hardcodes empty strings
R5	No server-side cart price validation	HIGH	**PARTIALLY RESOLVED**	Amount calculated server-side in batch creation, but createEPGPaymentSession doesn't re-validate the amount passed to it
R6	Non-atomic waitlist invite	MEDIUM	**OPEN**	DB write still happens before email send
R7	TOCTOU race on waitlist capacity	MEDIUM	**OPEN**	No advisory lock or stored procedure wrapping the check
R8	Cart holds live session references	MEDIUM	**OPEN**	No price snapshot at add-to-cart time
R9	No server-side soft hold (overselling)	MEDIUM	**OPEN**	No cart_holds table exists; no server-side reservation
R10	Session cloning no source reference	LOW	**OPEN**	No cloned_from_session_id column added
R11	Registration hold not enforced e2e	MEDIUM	**OPEN**	No hold-validity check at payment confirmation time
R12	DB trigger bug (RETURN NEW on DELETE)	LOW	**OPEN**	Trigger still uses RETURN NEW for DELETE operations
R13	Three divergent token-replacement implementations	LOW	**OPEN**	No shared utility in edge functions
R14	Confirmation email JSONB no version history	LOW	**OPEN**	No version/snapshot mechanism
R15	No optimistic locking on multi-admin editing	MEDIUM	**OPEN**	No updated_at check before writes

Score: 3 resolved, 1 partially resolved, 11 still open.

Part 2 — Current Threat Model

Who are the attackers?

Actor	Motivation	Access Level
Unauthenticated internet user	Curiosity, automated scanning, data theft	Can hit any public API route

Authenticated parent	Price manipulation, accessing other families' data	Has Supabase JWT, subject to RLS
Malicious admin	Unlikely but must be considered for compliance	Full admin access
Webhook spoofing	Fake payment confirmations, email status manipulation	Knows endpoint URLs

What are we protecting?

Asset	Sensitivity	Current Protection
Payment data (card last4, amounts, transaction IDs)	HIGH	EPG handles raw card data; local stores only masked data + audit JSON
Family PII (names, addresses, phone numbers, email)	HIGH	RLS on users/families/dancers tables
Registration data (who registered for what)	MEDIUM	RLS on registrations table
Admin credentials and session tokens	HIGH	Supabase Auth + httpOnly cookies
Media assets (images, PDFs)	LOW	**NO PROTECTION** — see V-1
Payment installment schedules	HIGH	**NO RLS** — see V-2

Part 3 — New Vulnerabilities (March 2026 Audit)

V-1 — Unauthenticated Media & Upload API Endpoints

Severity: CRITICAL
Status: RESOLVED (March 23, 2026)
Files: app/api/media/route.ts, app/api/media/[id]/route.ts, app/api/media/folders/route.ts, app/api/media/folders/[name]/route.ts, app/api/upload-image/route.ts, app/api/upload-pdf/route.ts

What was wrong

Six API endpoints accepted requests from **anyone on the internet** without checking authentication. They used inline serviceClient() functions with the Supabase **service role key** (bypassing all RLS).

Why this was a vulnerability

An attacker could list, delete, or upload files through predictable URLs (/api/media, /api/upload-image). The service role key is the Supabase equivalent of a database root password — using it for unauthenticated requests means zero access control.

Resolution

All 6 routes now:

1. Call requireAdmin() at the top of every handler — returns 401 if not authenticated as admin/super_admin
2. Use the shared createAdminClient() from utils/supabase/admin.ts instead of inline service clients (DRY)
3. The inline serviceClient() functions have been removed from all 6 files

---

V-2 — 9 Database Tables Without Row-Level Security

Severity: CRITICAL
Status: RESOLVED (March 23, 2026)
Location: Supabase PostgreSQL schema — supabase/migrations/20260323000001_enable_rls_nine_tables.sql

What was wrong

9 tables had no RLS, allowing any authenticated user (including parents) to read/write all data via the Supabase client library in the browser.

Tables that were unprotected

Table	What it contained	Risk if exposed
payments	Transaction IDs, amounts, raw EPG responses	Any user could read all payment data
batch_payment_installments	Installment schedules, amounts, due dates	Any user could modify installment data
class_sessions	Schedule, pricing, capacity, instructor names	Any user could modify class data
classes	Curriculum definitions, visibility settings	Any user could modify class visibility
class_requirements	Prerequisites and recommendation rules	Any user could bypass requirements
requirement_waivers	Audition/acceptance overrides	Any user could grant themselves waivers
semester_fee_config	Registration fees, costume fees, video fees	Any user could change fee amounts
session_occurrence_dates	Individual class dates, cancellation status	Any user could cancel class dates
tuition_rate_bands	Pricing tables by division and class count	Any user could change tuition prices

Resolution

Migration 20260323000001_enable_rls_nine_tables.sql implements:

1. **Instructor role support**: Added 'instructor' to users_role_check constraint, created is_instructor() helper function, added instructor_id UUID FK on class_sessions
2. **RLS enabled** on all 9 tables
3. **Policy tiers per table:**
    - admin/super_admin → full CRUD via is_admin_or_super()
    - service_role → full bypass (for webhooks, cron jobs, edge functions)
    - parent → scoped reads: own payment data (via registration_batches.parent_id), own waivers (via dancers.family_id), published catalog data
    - instructor → scoped reads: classes/sessions/occurrence dates they're assigned to (via class_sessions.instructor_id)
    - public catalog → SELECT on published semesters for registration flow (pricing, classes, sessions, fees, requirements, occurrence dates)
4. **TypeScript types updated**: User.role narrowed to "admin" | "super_admin" | "parent" | "instructor", ClassSession.instructor_id added

Note on instructor policies: Instructor-scoped policies are drafted based on best-guess access needs. The instructor_id column on class_sessions must be populated (either manually or via admin UI) for these policies to take effect. Refinement may be needed after client review.

V-3 — Client-Side-Only Admin Route Protection

Severity: HIGH
Status: RESOLVED (March 23, 2026)
Files: utils/supabase/middleware.ts, app/admin/layout.tsx

What was wrong

The admin section was protected only by a client-side useEffect check in app/admin/layout.tsx that redirected non-super_admin users. The middleware had a broken redirect condition (!pathname.startsWith("/") — always false) that never fired.

Why this was a vulnerability

Non-admin users could navigate directly to /admin/* routes and see SSR-rendered admin content before the client-side redirect fired. Server actions called from admin pages weren't protected by this layout check.

Resolution

1. **utils/supabase/middleware.ts**: Added server-side admin route protection inside updateSession(). For any /admin path:
    - Unauthenticated users → redirected to /auth
    - Authenticated users without admin or super_admin role → redirected to /
    - Cookies are preserved on redirect responses to prevent session desync
2. **app/admin/layout.tsx**: Fixed client-side check to accept both admin and super_admin (was only super_admin). Kept as defense-in-depth layer.
3. Removed the broken dead-code redirect condition that could never fire.

V-4 — Optional Webhook Signature Verification (Resend & Twilio)

Severity: HIGH (in production)
Status: RESOLVED (March 23, 2026)
Files: app/api/webhooks/resend/route.ts, app/api/webhooks/twilio/route.ts

What was wrong

Both webhook handlers had a conditional pattern: if the signature secret env var was set, verify; if not, silently process the unverified payload. A misconfigured deployment would accept forged webhooks.

Why this was a vulnerability

An attacker could send forged payloads to fake email delivery status or SMS delivery status, manipulating tracking data.

Resolution

Both handlers now **require** their respective secrets:

- **Resend**: If RESEND_WEBHOOK_SECRET is missing → returns 500 with "Webhook not configured" and logs an error. No fallback processing path.
- **Twilio**: If TWILIO_WEBHOOK_AUTH_TOKEN is missing → returns 500 with "Webhook not configured" and logs an error. No fallback processing path.

This matches the EPG webhook pattern which already required HTTP Basic auth credentials unconditionally.

V-5 — No Server-Side Re-Validation of Payment Amount

Severity: HIGH
Status: RESOLVED (March 23, 2026)
File: app/actions/createEPGPaymentSession.ts

What was wrong

createEPGPaymentSession() accepted amountDueNow from the client and passed it directly to EPG without checking it against the server-stored amount on the registration batch.

Why this was a vulnerability

A user could intercept the server action call and pass a modified amount (e.g., $1.00 instead of $500.00). The EPG order would be created for $1.00, the user would pay $1.00, and the webhook would confirm the batch.

Resolution

The server action now:

1. Fetches amount_due_now and grand_total from the registration_batches row (expanded the existing SELECT)
2. Compares the server-stored amount against the client-provided amount with a $0.01 tolerance
3. If mismatched, returns "Payment amount does not match. Please refresh and try again."
4. Uses the **server-stored amount** (not the client value) for both the EPG order creation and the payment record upsert

The amountDueNow input parameter is retained for the validation comparison but is never used as the authoritative amount.

V-6 — Weak Server Action Auth Pattern

Severity: HIGH
Files: app/admin/payments/actions/markInstallmentPaid.ts, app/actions/chargeStoredPaymentInstallment.ts

What's wrong

Some server actions check the caller's role and return an { error: "..." } object instead of throwing an exception. If the calling component doesn't check the return value, the action continues as if authorized.

Why this is a vulnerability

This is a defense-in-depth concern. If a developer calls markInstallmentPaid(id) and doesn't check the return:

```
// Missing error check — installment appears to be marked paid
await markInstallmentPaid(installmentId);
showSuccessToast("Marked as paid!");
```

The function would return { error: "Unauthorized" } but the UI would show success. While the database write wouldn't actually happen (the early return prevents it), this pattern is fragile and error-prone.

How to fix

Standardize on throwing errors for authorization failures:

```
const admin = await requireAdmin(); // throws if not admin
// ... proceed with action
```

V-7 — No Login Rate Limiting or Account Lockout

Severity: MEDIUM
File: app/auth/actions.ts

What's wrong

The login, signup, and password reset server actions have no rate limiting. The middleware rate limiter only covers /api/* routes, not server actions.

Why this is a vulnerability

- **Brute force**: An attacker can try thousands of password combinations per minute against known email addresses
- **Credential stuffing**: Automated tools can test leaked username/password pairs from other breaches
- **Account enumeration**: Password reset responds differently for valid vs invalid emails, allowing an attacker to build a list of registered email addresses
- **Spam accounts**: No CAPTCHA or email verification on signup means bots can create unlimited accounts

How to fix

1. Add rate limiting to auth server actions (use Upstash — already in the project):

   ```
   const limiter = getAuthLimiter(); // 5 attempts per 15 min per IP
   const { success } = await limiter.limit(ip);
   if (!success) return { error: "Too many attempts. Try again later." };
   ```

2. Return the same response for valid and invalid emails on password reset
3. Add email verification on signup (Supabase supports this natively)
4. Consider adding CAPTCHA for public-facing forms

V-8 — No File Magic Byte Validation on Uploads

Severity: MEDIUM
Files: app/api/upload-image/route.ts, app/api/upload-pdf/route.ts

What's wrong

Upload endpoints validate the MIME type reported by the browser (file.type) but do not verify the actual file content by checking magic bytes (the first few bytes of a file that identify its true format).

Why this is a vulnerability

MIME types are set by the browser based on file extension and are trivially spoofable. An attacker could:

1. Rename a ZIP, EXE, or HTML file to have a .jpg extension
2. The browser reports image/jpeg as the MIME type
3. The upload endpoint accepts it
4. The file is stored in Supabase storage and served with a public URL
5. If it's HTML, it could contain JavaScript that executes in the context of your domain (XSS via stored file)

How to fix

Validate magic bytes using the file-type library (or check the first bytes manually):

```
import { fileTypeFromBuffer } from 'file-type';
const type = await fileTypeFromBuffer(buffer);
if (!type || !['image/jpeg', 'image/png', 'image/gif', 'image/webp'].includes(type.mime)) {
  return NextResponse.json({ error: "Invalid file type" }, { status: 400 });
}
```

Part 4 — Carried-Forward Structural Risks (Still Open)

V-9 — Broadcast Email Variable Resolution Still Broken

Severity: HIGH (unchanged from Feb audit)
File: supabase/functions/send-email-broadcast/index.ts:118-120

The broadcast function still hardcodes student_name, semester_name, and session_name as empty strings. Any broadcast email template using these tokens renders blank fields silently.

Why this matters: The admin has a UI to insert these tokens. They appear to work in preview. They silently fail on send. This will be visible to every recipient of every broadcast email that uses those tokens.

Fix: Pass semester/session context from the email's recipient selection into the variable map.

V-10 — No Server-Side Cart Hold (Overselling Risk)

Severity: MEDIUM (unchanged)

Cart items exist only in localStorage. No server-side reservation is created. 30 users can simultaneously add the last 2 spots to their carts, proceed through registration, and 28 will fail at an undefined point.

Why this matters for the client audit: The client will ask "what happens if two families try to register for the last spot at the same time?" The answer right now is "undefined behavior."

Fix: Create a cart_holds table. When checkout begins, reserve capacity server-side with a TTL. Use SELECT ... FOR UPDATE SKIP LOCKED to prevent double-assignment.

V-11 — Non-Atomic Waitlist Invite

Severity: MEDIUM (unchanged)

The waitlist writes status='invited' to the database before attempting the email send. If the edge function crashes between the DB write and the email send, the entry is stuck as "invited" with no email sent, blocking the entire session's waitlist queue for up to 48 hours.

Why this matters: This is a silent failure. The admin sees "invited" and assumes the email went out. The parent never received anything. The next person in line is blocked.

Fix: Attempt the email send first, then write to DB on success. The email without the DB write is harmless; the DB write without the email is stuck.

V-12 — Registration Hold Not Enforced at Payment Confirmation

Severity: MEDIUM (unchanged)

When the EPG webhook confirms a payment, it creates registrations without checking whether the original hold_expires_at has passed. If a user's 30-minute hold expires while they're on the EPG payment page, the hold is released by the cron job, the capacity may be given to a waitlisted user, and then the original user's payment completes — creating a registration that exceeds capacity.

Why this matters: This is a race condition that gets worse under load. It can result in classes having more students than their stated capacity.

Fix: In confirmBatch(), before creating registration rows, verify that capacity is still available. If not, void the payment and notify the user.

V-13 — TOCTOU Race on Waitlist Capacity Check

Severity: MEDIUM (unchanged)

The capacity check and invite issuance are separate, non-atomic steps with no lock held between them.

Fix: Wrap in a PostgreSQL advisory lock or stored procedure with SELECT ... FOR UPDATE on the session row.

V-14 — No Optimistic Locking for Multi-Admin Editing

Severity: MEDIUM (unchanged)

If two admins edit the same semester simultaneously, the last save silently overwrites the first with no conflict detection.

Fix: Add updated_at check before persistSemesterDraft writes. Return a conflict error if another write occurred since the current user loaded the form.

V-15 — Three Divergent Token-Replacement Implementations

Severity: LOW (unchanged)

File	Behavior on unknown token
utils/resolveEmailVariables.ts	Preserves {{key}} — safe
supabase/functions/send-email-broadcast/index.ts	Replaces with empty string — data-corrupting
supabase/functions/process-waitlist/index.ts	replaceAll loop — no fallback

Fix: Create shared utility at supabase/functions/_shared/resolveTokens.ts.

V-16 — DB Trigger Returns RETURN NEW on DELETE

Severity: LOW (unchanged)

The prevent_child_modification_if_semester_published() trigger returns RETURN NEW for DELETE operations instead of RETURN OLD, which blocks deletion even when it should be allowed (published semester with 0 registrations).

Fix: Change to RETURN OLD in the DELETE branch.

Part 5 — EPG / PCI-DSS Compliance Assessment

Scope

In scope (your code handles this):

- Webhook endpoint receiving payment notifications
- Server actions initiating payment sessions
- Stored payment method pointers (masked data only)
- Installment charging logic

Out of scope (Elavon handles this):

- Card data collection (hosted payment page)
- Card storage (EPG Stored Cards/ACH)
- PCI-certified gateway infrastructure

Compliance Strengths

Requirement	Status	Evidence
3.2.1 — PAN unreadable	Compliant	No PANs stored; only masked_number + last4
6.5.10 — Broken auth		

prevention	Compliant	HTTP Basic auth + timing-safe comparison on webhook
8.1.1 — Unique user IDs	Compliant	Users linked to Supabase auth.users; shoppers linked to user_id
10 — Logging/monitoring	Compliant	raw_notification + raw_transaction stored as audit trail

Compliance Gaps

Requirement	Status	Action Needed
3.4 — Encryption at rest	Unknown	Verify Supabase encryption at rest is enabled for the database
6.2 — Security patches	Gap	Add dependency scanning (Dependabot or npm audit in CI)
7.1 — Limit access	**Compliant**	payments table now has RLS with admin-only write + parent-scoped read (V-2 resolved)
11.2 — Vulnerability scanning	Gap	No automated vulnerability scanning configured

EPG-Specific Security Observations

Strong practices:

- Webhook validates HTTP Basic auth with crypto.timingSafeEqual() — prevents timing oracle attacks
- Never trusts webhook notification payload — always re-fetches authoritative transaction state from EPG API
- Idempotency guards: checks payment terminal state before processing (replay-safe)
- Server-only credentials: EPG_SECRET_KEY and EPG_MERCHANT_ALIAS never in client bundle
- Stored method HREFs (pointers) only — no card data in application database

Items to discuss with Elavon developer:

- createEPGPaymentSession should not accept amount from client (see V-5)
- Confirm EPG webhook retry policy matches your idempotency design
- Verify that Elavon's Self-Assessment Questionnaire (SAQ A or A-EP) covers the hosted payment page model
- Discuss PCI 4.0 requirements if applicable to your merchant level

---

Part 6 — Production Readiness Checklist

Must-Fix Before Client Audit (~April 2) — ALL RESOLVED

- [x] **V-1**: Add authentication to all 6 media/upload API endpoints *(resolved 2026-03-23)*
- [x] **V-2**: Enable RLS on 9 unprotected tables with 3-tier role policies *(resolved 2026-03-23)*
- [x] **V-3**: Add server-side admin route check in middleware *(resolved 2026-03-23)*
- [x] **V-4**: Require webhook signature secrets in production *(resolved 2026-03-23)*
- [x] **V-5**: Re-validate payment amount from batch, not client input *(resolved 2026-03-23)*

Must-Fix Before Production Launch

- [ ] **V-6**: Standardize server action auth to throw, not return errors
- [ ] **V-7**: Add login rate limiting and consistent password reset response

- [ ] **V-8**: Add file magic byte validation on uploads
- [ ] **V-9**: Fix broadcast email variable resolution (empty strings)
- [ ] **V-10**: Add server-side cart hold mechanism (even a basic capacity check at checkout start)
- [ ] **V-12**: Add hold_expires_at validation in confirmBatch()
- [ ] Set RESEND_WEBHOOK_SECRET and TWILIO_WEBHOOK_AUTH_TOKEN in production env
- [ ] Set ADMIN_NOTIFICATION_EMAIL in production env
- [ ] Run npm audit and resolve critical/high severity findings
- [ ] Verify Supabase encryption at rest is enabled
- [ ] Confirm EPG environment variables point to production (not UAT)
- [ ] Set up Twilio A2P 10DLC brand + campaign registration

Should-Fix Post-Launch

- [ ] **V-11**: Invert waitlist invite ordering (email before DB write)
- [ ] **V-13**: Add advisory lock to waitlist capacity check
- [ ] **V-14**: Add optimistic locking to semester editor
- [ ] **V-15**: Consolidate token-replacement implementations
- [ ] **V-16**: Fix DB trigger RETURN NEW → RETURN OLD on DELETE
- [ ] Add email verification on signup
- [ ] Add CAPTCHA to public-facing forms
- [ ] Add audit logging for all sensitive mutations (payments, deletions, role changes)
- [ ] Implement parent-facing stored payment method management UI
- [ ] Add parent notification when installment 1 charge declines
- [x] Add instructor RLS role *(drafted 2026-03-23 — needs instructor_id population and policy refinement)*

Part 7 — Scalability Assessment

What Breaks First at 10x Load

Bottleneck	Current Behavior	Breaking Point	Mitigation
Email broadcast	100 concurrent Resend calls per batch	50k+ recipients → Resend rate limits	Per-batch delays + retry queue
Waitlist cron	Sequential session processing	100+ sessions → overlapping cron cycles	Parallel processing with Promise.allSettled()
Draft persistence	Full sync on every step navigation	20 sessions × 30 days = 600+ rows per save	Dirty-check + debounce
Multi-admin editing	Last writer silently wins	2+ admins → data loss	Optimistic locking
Installment charging	Daily cron, serial per installment	500+ overdue → timeout	Parallel batch processing
resolveRecipients()	Multi-table join	10k+ users → slow query	Indexes + materialized view

What Becomes Unsafe First

- **Discount stacking**: No floor enforcement. Two stacking discounts could produce a negative total.
- **Payment deduplication**: Browser double-submission could create duplicate registrations without an idempotency key on registration creation.
- **Cart race condition**: Without server-side holds, concurrent checkouts will oversell.

Part 8 — Summary Risk Table

ID	Severity	Finding	Status	Location
V-1	**CRITICAL**	Unauthenticated media/upload API endpoints	**RESOLVED**	app/api/media/, app/api/upload-*
V-2	**CRITICAL**	9 database tables without RLS	**RESOLVED**	supabase/migrations/20260323000001_*.sql
V-3	**HIGH**	Client-side-only admin route protection	**RESOLVED**	utils/supabase/middleware.ts, app/admin/layout.tsx
V-4	**HIGH**	Optional webhook signature verification	**RESOLVED**	webhooks/resend, webhooks/twilio
V-5	**HIGH**	No server-side re-validation of payment amount	**RESOLVED**	createEPGPaymentSession.ts
V-6	**HIGH**	Weak server action auth (returns error, doesn't throw)	OPEN	markInstallmentPaid.ts, chargeStoredPaymentInstallment.ts
V-7	**MEDIUM**	No login rate limiting or account lockout	OPEN	app/auth/actions.ts
V-8	**MEDIUM**	No file magic byte validation	OPEN	upload-image/route.ts, upload-pdf/route.ts
V-9	**HIGH**	Broadcast email variables still hardcoded empty	OPEN (Feb)	send-email-broadcast/index.ts:118-120
V-10	**MEDIUM**	No server-side cart hold — overselling risk	OPEN (Feb)	CartProvider.tsx
V-11	**MEDIUM**	Non-atomic waitlist invite	OPEN (Feb)	process-waitlist/index.ts
V-12	**MEDIUM**	Registration hold not enforced at payment confirmation	OPEN (Feb)	EPG webhook handler
V-13	**MEDIUM**	TOCTOU race on waitlist capacity	OPEN (Feb)	process-waitlist/index.ts
V-14	**MEDIUM**	No optimistic locking on semester editing	OPEN (Feb)	SemesterForm.tsx
V-15	**LOW**	Three divergent token-replacement implementations	OPEN (Feb)	Edge functions
V-16	**LOW**	DB trigger RETURN NEW on DELETE	OPEN (Feb)	Supabase trigger

Total: 8 resolved (3 Feb + 5 March), 11 still open.

Part 9 — Environment Security Checklist

Before Going to Production

Check	Status
.env.local not committed to git	Verify
No NEXT_PUBLIC_* variables contain secrets	Verify

SUPABASE_SERVICE_ROLE_KEY never used in client components	Verified
EPG credentials (EPG_SECRET_KEY, webhook creds) only in server code	Verified
Twilio credentials only in server code	Verified
Vercel environment variables set for production	Configure
Supabase project is on a paid plan (for edge function invocations)	Verify
pg_cron jobs configured in production Supabase	Configure
Resend domain verified and DNS records set	Configure
Twilio A2P 10DLC registration complete	Pending (~1 week)
Custom domain configured on Vercel	Configure
SSL/TLS on custom domain	Automatic with Vercel

Audit date: March 22, 2026
System: AYDT Registration Admin Portal
Audit type: Security + vulnerability + production readiness
Previous audit: February 24, 2026 (original), March 16, 2026 (update)